



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

CPS 3962 Objected-Oriented Analysis and Design

Final Project 2: Online Meal Delivery System

Dr. Hamza Djigal

Assistant Professor

Department of Computer Science, Wenzhou-Kean University

dhamza@kean.edu

March 26, 2026 (Spring)

Contents

1	Project Overview	2
2	Waterfall Methodology Phases	2
2.1	Requirements Gathering	2
2.2	System Analysis	3
2.3	System Design	3
2.3.1	Database Schema (Example)	3
2.3.2	UI Wireframes (Placeholders)	4
2.4	4. Implementation	4
2.5	5. Testing	5
2.6	6. Deployment (optinal but good as prototype	5
2.7	7. Documentation & Final Submission	6
3	Optional Advanced Features (Extra Credit)	6

1 Project Overview

Project Name: SmartMeal – Online Meal Delivery Platform

Objective: Build a platform connecting customers, restaurants, and delivery drivers with real-time order tracking, payment, and rating functionalities.

Key Features (MVP):

- User Management: Customer, Restaurant, and Delivery Driver registration, login, profile management.
- Menu Management: Restaurants can manage menu items, prices, and availability.
- Order Placement & Tracking: Customers place orders and track delivery in real-time.
- Payment Integration: Online payment, cash on delivery, promo codes, discounts.
- Ratings & Feedback: Customers rate food and delivery; optional restaurant ratings.
- Admin Panel: Monitor users, restaurants, orders, payments, and complaints.

2 Waterfall Methodology Phases

2.1 Requirements Gathering

- Functional Requirements:
 - Customer: Register, browse restaurants, order food, track delivery, pay, rate.
 - Restaurant: Manage menu, receive orders, update order status.
 - Delivery Driver: Accept delivery, update location, deliver order.
 - Admin: Monitor platform, manage users/restaurants/drivers, generate reports.
- Non-Functional Requirements:
 - Real-time notifications for order and delivery status.
 - Secure payment processing.
 - Scalable system to support multiple restaurants and users.
- Deliverables:
 - Use Case Diagram (Customer, Restaurant, Driver, Admin)
 - Use Case Descriptions (preconditions, main flow, alternate flows)

2.2 System Analysis

Placeholder Diagrams: Students to replace with actual diagrams.

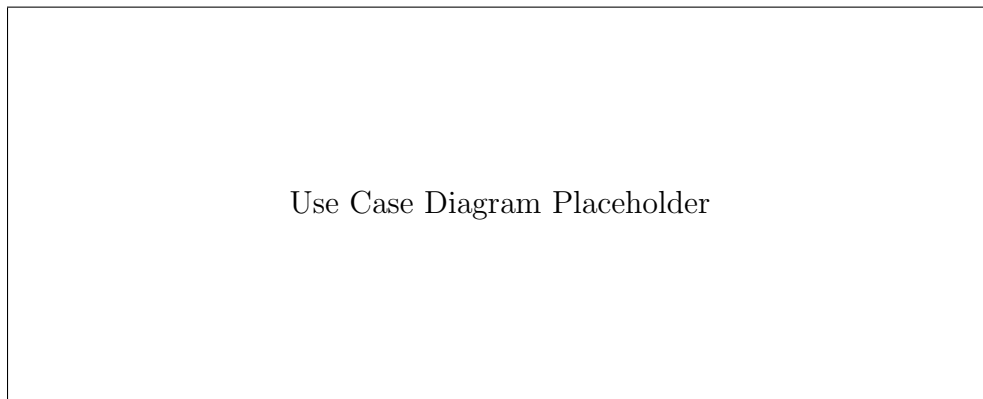


Figure 1: Use Case Diagram (Customer, Restaurant, Driver, Admin)

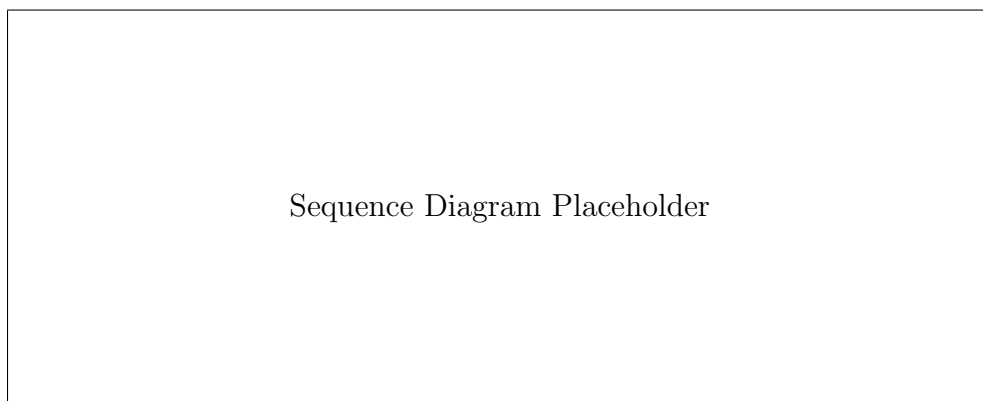


Figure 2: Sequence Diagram: Placing an Order

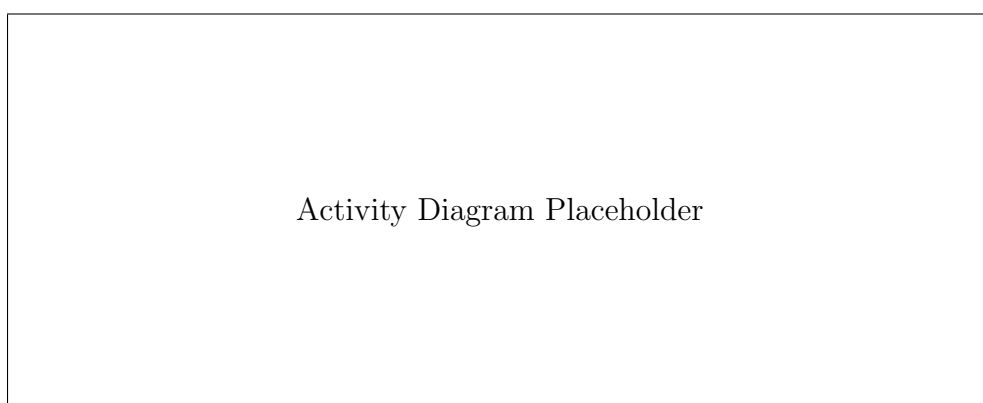


Figure 3: Activity Diagram: Order Processing and Delivery Flow

2.3 System Design

2.3.1 Database Schema (Example)

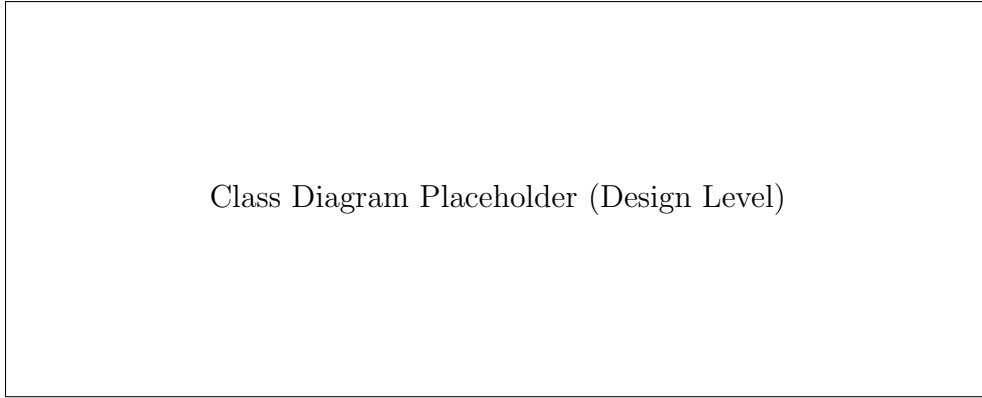


Figure 4: Class Diagram: User, Customer, Restaurant, Driver, Order, Menu, Payment

Table	Columns	Data Type	Description
Users	user_id (PK), name, email, password, role	INT, VARCHAR	Customer, Restaurant, Driver info
Restaurants	restaurant_id (PK), name, location, rating	INT, VARCHAR, VARCHAR, FLOAT	Restaurant profile
MenuItems	item_id (PK), restaurant_id (FK), name, price, availability	INT, INT, VARCHAR, FLOAT, BOOLEAN	Restaurant menu items
Orders	order_id (PK), customer_id (FK), driver_id (FK), restaurant_id (FK), status, total_amount, order_time, delivery_time	INT, INT, INT, INT, VARCHAR, FLOAT, DATETIME, DATETIME	Order details
Payments	payment_id (PK), order_id (FK), amount, method, status	INT, INT, FLOAT, VARCHAR, VARCHAR	Payment records
Ratings	rating_id (PK), order_id (FK), rating, comments	INT, INT, INT, TEXT	Customer; Driver; Restaurant feedback

2.3.2 UI Wireframes (Placeholders)

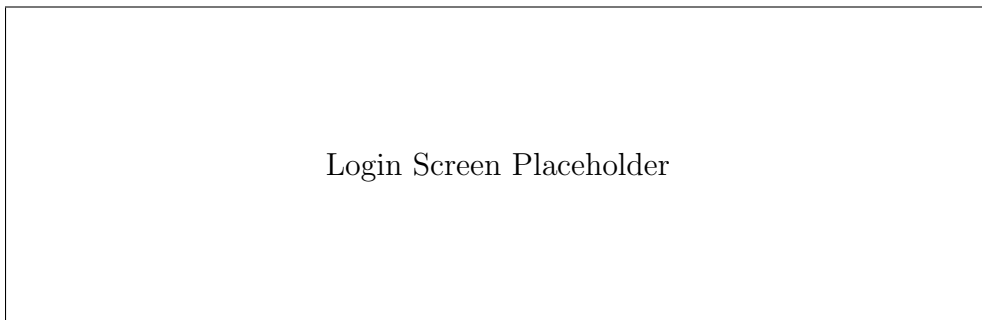


Figure 5: Login Screen

2.4 4. Implementation

4

- Modules:

1. Authentication & User Management

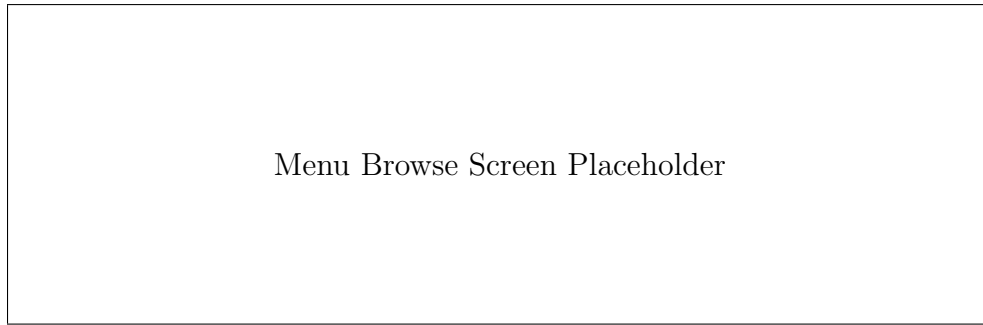


Figure 6: Menu Browse Screen



Figure 7: Order Tracking Screen

- Follow OO principles: encapsulation, inheritance, polymorphism
- Version control with GitHub/GitLab

2.5 5. Testing

- Unit Testing for each module/class
- Integration Testing: Order, payment, and tracking flow
- System Testing: Complete platform functionality
- Acceptance Testing based on use cases
- Deliverables:
 - Test Plan
 - Test Cases (expected vs actual results)
 - Bug Tracking Report

2.6 6. Deployment (optinal but good as prototype)

- Backend deployment: AWS / Heroku
- Frontend deployment: Web (Netlify) / Mobile (APK / iOS TestFlight)

- Deployment Guide documentation

2.7 7. Documentation & Final Submission

- Requirements Specification
- UML Diagrams (Use Case, Class, Sequence, Activity)
- Database Schema
- UI Screens / Wireframes
- Test Plan & Results
- Deployment Guide
- Live Demo of MVP platform

3 Optional Advanced Features (Extra Credit)

- AI-based food recommendations
- Dynamic delivery fee calculation
- Multi-language support
- Customer-to-restaurant/driver chat
- Loyalty rewards and referral system

Suggested Tech Stack:

- Frontend: ReactJS / Flutter
- Backend: Node.js / Django / Java Spring Boot
- Database: PostgreSQL / MySQL
- Maps & Tracking: Google Maps API / OpenStreetMap
- Deployment: AWS / Heroku / Docker